

Обраний варіант.

Для виконання лабораторної роботи було обрано такі пункти з переліків:

з першого переліку було обрано пункт №2 (Список. Реалізації на основі зв'язних списків, масивів, бібліотечних засобів. Алгоритми сортування: insertion sort, quicksort, merge sort.) та з другого переліку було обрано пункт №1 (Математичний аналіз.).

Виконане завдання відповідно обраного варіанту роботи.

У рамках роботи було реалізовано такі класи:

1. Реалізація списків

1.1. Було реалізовано абстрактний клас `BaseList` для узагальнення роботи з списками чотирьох типів. Це дає реалізувати механізм легкого підключення нових алгоритмів, адже потрібно лише створити новий клас, що успадковується від даного класу та має всі потрібні методи.

1.2. Клас `ArrayList`. Клас що успадковується від базового та є реалізацією списків на масивах.

1.2.1. Поля: 3 поля

1.2.1.1. `_data` (масив для зберігання даних списку)

1.2.1.2. `capacity` (максимальна кількість елементів у списку)

1.2.1.3. `_count` (об'єм списку)

1.2.2. Нетривіальні методи: 7

1.2.2.1. `__iter__` (ітератор для можливості працювати з списком як з ітерованим об'єктом, тобто зручно працювати з циклами)

1.2.2.2. `_resize` (допоміжний метод)

1.2.2.3. `add` (додає елемент до списку у кінець)

1.2.2.4. `remove` (видаляє елемент списку за заданим індексом)

1.2.2.5. `swap` (переставляє місцями два елементи)

- 1.2.2.6. max (знаходить максимальний елемент)
 - 1.2.2.7. min (знаходить мінімальний елемент у списку)
 - 1.3. Клас LinkedList. Клас що успадковується від базового абстрактного та є реалізацією однозв'язних списків.
 - 1.3.1. Поля: 1 поле
 - 1.3.1.1. head (вказівник на голову (перший елемент) списку)
 - 1.3.2. Нетривіальні методи: 7 (аналогічно до списку на масивах)
 - 1.4. Клас DoublyLinkedList. Успадковується від класу з реалізацією зв'язних списків, є реалізацією двозв'язних списків.
 - 1.4.1. Поля: 2
 - 1.4.1.1. head (вказівник на голову (перший елемент) списку)
 - 1.4.1.2. tail (вказівник на хвіст (останній елемент) списку)
 - 1.4.2. Нетривіальні методи: 7 (аналогічно до попередніх списків)
 - 1.5. Клас Node. Клас для вузла зв'язного списку.
 - 1.5.1. Поля: 2
 - 1.5.1.1. data (зберігає дані у вузлі)
 - 1.5.1.2. next (вказівник на наступний елемент)
 - 1.6. Клас DoublyNode. Клас, що успадковується від класу вузла та потрібний для реалізації двозв'язних списків.
 - 1.6.1. Поля: 3
 - 1.6.1.1. data (поле для зберігання даних у вузлі)
 - 1.6.1.2. next (вказівник на наступний елемент)
 - 1.6.1.3. prev (вказівник на попередній елемент)
 - 1.7. Клас LibraryList. Клас реалізований на вбудованих методах.
 - 1.7.1. Поля: 1
 - 1.7.1.1. data (зберігання даних у бібліотечному списку)
- 2. Алгоритми сортування.
 - 2.1. Клас SortingAlgorithm. Потрібний для зручного підключення нових алгоритмів сортування, аже для цього потрібно створити лише клас для нового алгоритму, що матиме метод sort.
 - 2.2. Клас InsertionSort. Клас для сортування вставкою.
 - 2.2.1. Поля: без полів.
 - 2.2.2. Нетривіальні методи: 1

- 2.2.2.1. `sort()`
- 2.3. Клас `QuickSort`.
 - 2.3.1. Поля: без полів.
 - 2.3.2. Нетривіальні методи: 3
 - 2.3.2.1. `partition`
 - 2.3.2.2. `_quick_sort` (метод з швидким сортуванням)
 - 2.3.2.3. `sort` (метод обгортка для методу з швидким сортуванням, створений для зручності та відповідності абстрактному класу)
- 2.4. Клас `MergeSort`.
 - 2.4.1. Поля: без полів
 - 2.4.2. Нетривіальні методи: 3
 - 2.4.2.1. `_merge`
 - 2.4.2.2. `_merge_sort` (метод для сортування злиттям)
 - 2.4.2.3. `sort` (метод-обгортка для методу з сортуванням злиттям, створений для зручності та відповідності абстрактному класу)
- 2.5. Клас `SelectionSort`.
 - 2.5.1. поля: без полів
 - 2.5.2. Нетривіальні методи: 1
 - 2.5.2.1. `sort` (метод з програмною реалізацією сортування вибором)
- 2.6. Клас `BubbleSort`.
 - 2.6.1. Поля: без полів
 - 2.6.2. Нетривіальні методи: 1
 - 2.6.2.1. `sort`
- 2.7. Клас `ThreeWayMergeSort`.
 - 2.7.1. Поля: клас не має полів
 - 2.7.2. Нетривіальні методи: 3
 - 2.7.2.1. `_merge` (метод для злиття трьох підсписків)
 - 2.7.2.2. `_three_way_merge` (метод з програмною реалізацією трифазного сортування злиттям)
 - 2.7.2.3. `sort` (метод-обгортка для методу з трифазним сортуванням злиттям, створений для зручності використання та відповідності батьківському абстрактному класу)

- 2.8. Клас BucketSort.
 - 2.8.1. Поля: клас не має полів
 - 2.8.2. Нетривіальні методи: 1
 - 2.8.2.1. sort
- 2.9. Клас CountSort.
 - 2.9.1. Поля: клас не має полів.
 - 2.9.2. Нетривіальні методи: 1
 - 2.9.2.1. sort
- 3. Робота з системою Sage
 - 3.1. Клас SageRemote
 - 3.1.1. Поля: 2
 - 3.1.1.1. url (посилання, куди відправляється запит)
 - 3.1.1.2. timeout (час очікування)
 - 3.1.2. Нетривіальні методи: 1
 - 3.1.2.1. run_code (відправка запиту на обчислення)
- 4. Реалізація пункту про математичний аналіз.
 - 4.1. Клас Function. Робота з функціями
 - 4.1.1. Поля: 3
 - 4.1.1.1. _expression (формула, якою задається функція)
 - 4.1.1.2. _variables (список змінних)
 - 4.1.1.3. _list_type (тип списку, з яким йде робота)
 - 4.1.2. Нетривіальні методи: 13
 - 4.1.2.1. evaluate (обчислення значення функції в точці)
 - 4.1.2.2. _is_increasing (виявлення зростання функції)
 - 4.1.2.3. _is_decreasing (виявлення спадання функції)
 - 4.1.2.4. is_monotonic (виявлення монотонності функції)
 - 4.1.2.5. is_bounded (виявлення обмеженості функції)
 - 4.1.2.6. approximate_limit (обчислення приблизного ліміту в точці)
 - 4.1.2.7. sym_limit (обчислення ліміту за допомогою Sage)
 - 4.1.2.8. derivative (обчислення похідної)
 - 4.1.2.9. partial_derivative (обчислення похідної за якоюсь з змінних, якщо таких декілька)
 - 4.1.2.10. gradient (обчислення вектору з часткових похідних)
 - 4.1.2.11. integral (обчислення невизначеного інтеграла)
 - 4.1.2.12. plot (побудова графіка)
 - 4.1.2.13. export_to_json (запис даних по функції у файл)

4.2. Клас Sequence. Клас для роботи з послідовностями.

4.2.1. Поля: 3

4.2.1.1. `_expression` (формула, якою задається функція)

4.2.1.2. `_variables` (список змінних)

4.2.1.3. `_list_type` (тип списку, з яким йде робота)

4.2.2. Нетривіальні методи: 8

4.2.2.1. `evaluate` (обчислення ен-того члена послідовності)

4.2.2.2. `_is_increasing` (чи є послідовність зростаючою)

4.2.2.3. `_is_decreasing` (чи є спадною)

4.2.2.4. `is_monotonic` (чи є монотонною)

4.2.2.5. `is_bounded` (чи обмежена)

4.2.2.6. `approximate_limit` (ліміт, що обчислюється
приблизно)

4.2.2.7. `sym_limit` (ліміт, що обчислюється за допомогою
`sage`)

4.2.2.8. `export_to_json` (запис результатів по послідовності
у файл)

4.3. Клас Series. Клас для роботи з рядами.

4.3.1. Поля: 3

4.3.1.1. `_expression` (формула, якою задається функція)

4.3.1.2. `_variables` (список змінних)

4.3.1.3. `_list_type` (тип списку, з яким йде робота)

4.3.2. Нетривіальні методи: 3

4.3.2.1. `term` (обчислення ен-того члену ряду)

4.3.2.2. `is_convergent` (чи є ряд збіжним)

4.3.2.3. `taylor_series` (розклад функції в ряд тейлора)

Весь функціонал консольної програми було реалізовано у файлі
[main.py](#).

Загальна кількість класів: 20

Загальна кількість полів: 23

Загальна кількість нетривіальних методів: 60

Було реалізовано 3 ієрархії успадкування, при тому клас для роботи з
двухв'язними списками успадковується від класу для роботи з
однов'язними списками, тобто ця лінія містить три класи в ієрархії.

Поліморфізм проявляється у декількох місцях. Передусім, це
реалізація різних типів списків (`ArrayList`, `LinkedList`, інші), які

успадковуюють абстрактний клас `BaseList` і перевизначають його методи — це приклад динамічного поліморфізму, оскільки конкретна реалізація методів (`add`, `remove`, `get`, `size`) обирається під час виконання залежно від типу списку, який було створено. Другим прикладом є модуль сортування: абстрактний інтерфейс сортувальника (`sort`) реалізується різними класами (`InsertionSort`, `QuickSort`, `MergeSort`, `BubbleSort`, `SelectionSort`, `CountSort`), які мають спільний метод `sort`, але виконують різні алгоритми. Виклик `sorter.sort(target_list)` є динамічним поліморфізмом, оскільки конкретний алгоритм визначається вибором користувача під час виконання програми. Крім того, певний елемент статичного поліморфізму проявляється у використанні параметрів за замовчуванням у методах (наприклад, `is_monotonic`, `approximate_limit`, `is_bounded`), що дозволяє викликати функції у різних варіаціях без створення окремих перевантажених методів.

Завдання з першого списку.

1. (**) Список. Реалізації на основі зв'язних списків, масивів, бібліотечних засобів. Алгоритми сортування: `insertion sort`, `quicksort`, `merge sort`.
 - a. +* за інші алгоритми сортування;
 - b. +* за реалізацію швидких алгоритмів, не заснованих на порівнянні (`non-comparison sorts`, наприклад `counting`, `radix`, `bucket sort`);
 - c. +* за реалізацію загального механізму, що дозволить підключати нові алгоритми сортування;
 - d. +* за підтримку різних способів сортування (ключів сортування, зростання/спадання) для одного об'єкта.
 - e. +* за підтримку сортування за кількома критеріями (наприклад, спершу за одним полем, якщо значення однакові — за іншим і т.д.)

Було виконано завдань на 5 зірочок. Зеленим виділено завдання які були виконані.

Завдання з другого списку.

1. Математичний аналіз.

- a. Послідовності, функції (зокрема декількох змінних). Ліміти, похідні, інтеграли, суми рядів.
- b. Алгоритми обчислення похідних (для довільних функцій), а також в окремих випадках інтегралів, сум рядів, лімітів. Розкладення функцій в ряди.
- c. Інтеграція з системами комп'ютерної математики.

На рівні a було реалізовано класи, що описують об'єкти та методи, що отримують дані та змінюють їх. На рівні b було реалізовано нетривіальні алгоритми для обчислення похідних, сум рядів, границь, розкладення функцій в ряди. Було реалізовано методи для роботи з рядами та послідовностями. На рівні c було реалізовано роботу з зовнішньою роботою екомп'ютерної математики Sage, за допомогою допоміжного класу. Виконані роботи за рівнями можна побачити в коді програми, адже вони розділені коментарями, що підкреслюють рівень, для якого було зроблено цей метод чи функцію.

Бібліотеки

Для виконання роботи/проєкту було використано додаткові бібліотеки: matplotlib, numpy, requests.

Звіт по роботі з matplotlib.

Плануючи роботу, я використовувала бібліотеку matplotlib для розв'язання задач, пов'язаних із побудовою графіків функцій, візуалізацією даних та представленням результатів обчислень у наочній графічній формі. Основна мета полягала у можливості швидко отримувати лінійні графіки, діаграми та інші прості типи візуалізації для перевірки правильності реалізованих алгоритмів.

Я обрала саме matplotlib, оскільки це найбільш часто використовувана бібліотека для побудови графіків у Python. Вона добре документована, стабільна та має широкий набір функцій. Інші альтернативи, як-от seaborn чи plotly, хоча й мають свої переваги, є або

надбудовами над `matplotlib`, або орієнтовані на складнішу інтерактивність, яка мені не була потрібна.

Отримання та встановлення бібліотеки пройшло без будь-яких труднощів: вона доступна через `pip` і встановлюється за декілька секунд. Почати використовувати `matplotlib` теж було нескладно — базові приклади з документації дозволили швидко розібратися з основними інструментами побудови графіків.

Документація виявилася доволі об'ємною, але структурованою. Вона містить багато прикладів, пояснень та зображень, що робить її корисною для практичного застосування. Окремі моменти можуть здатися перевантаженими, однак загалом вона відповідає очікуванням.

Розібратися, які саме функції та класи потрібні для побудови конкретного графіка, було нескладно. Інтерфейс бібліотеки логічний: достатньо створити фігуру, вибрати вісь та побудувати дані. Частина функцій має дещо громіздкі назви чи параметри, але це не заважало роботі.

Використовувати бібліотеку було зручно. `Matplotlib` не вимагала написання надлишкового коду, хоча інколи для досягнення бажаного вигляду графіка доводилось додавати додаткові налаштування. Поведінка функцій була передбачуваною, а взаємодія між різними частинами бібліотеки загалом зрозумілою.

Проблеми виникали лише при спробах налаштування вигляду графіків, оскільки деякі параметри впливають один на одного неочевидним чином.

Позитивні сторони `matplotlib` полягають у її стабільності, універсальності та широких можливостях. Вона підходить як для найпростішої візуалізації, так і для складних професійних графіків. Негативним аспектом можна вважати необхідність налаштовувати зовнішній вигляд графіків вручну, що іноді займає більше часу, ніж хотілося б.

Звіт по роботі з numpy.

Під час реалізації модуля для роботи з математичними функціями я використовувала бібліотеку NumPy, головним чином для генерації рівномірно розподілених числових вибірок у методі `plot()`. Основною задачею було забезпечити ефективне та зручне формування масиву точок у заданому діапазоні, які надалі застосовувалися для побудови графіка функції.

Вибір NumPy був логічним, оскільки ця бібліотека є стандартом у Python для чисельних обчислень, забезпечує високу продуктивність та містить готові інструменти для роботи з масивами. Аналоги на кшталт стандартного `range()` були менш зручними, а використання спискових включень для генерації чисел ускладнювало б подальшу взаємодію з Matplotlib, який також очікує дані у форматі масивів.

Документація NumPy добре структурована, але для конкретних задач, які потрібно було вирішити в межах проєкту, достатньо було базових прикладів використання `numpy.linspace()` та перетворення списків у масиви.

Використання бібліотеки виявилось зручним, оскільки дозволило уникнути написання додаткового коду для генерації числової сітки та забезпечило сумісність з Matplotlib. Робота функцій була передбачуваною та стабільною, без неочікуваних побічних ефектів. Взаємодія з рештою мого коду була простою: NumPy використовувалася лише для створення наборів даних, а вся логіка обчислень залишалася на стороні класу `Function`.

Проблем при використанні NumPy не виникло. Бібліотека працювала коректно, інтеграція з іншими частинами проєкту не створювала труднощів.

До позитивних сторін можна віднести простоту використання та відмінну сумісність з іншими інструментами для наукових обчислень. Негативних аспектів у межах виконуваних задач я не виявила, оскільки бібліотека повністю закривала ті потреби, для яких її було залучено.

Звіт по роботі з бібліотекою requests.

У проєкті я застосовувала бібліотеку Requests для реалізації віддаленого виконання коду в службі SageMathCell у класі SageRemote. Основною задачею було надійно надсилати HTTP-запити до сервера, передавати код у форматі JSON і отримувати результат виконання. Requests була вибрана тому, що вона є фактичним стандартом для роботи з HTTP у Python: проста у використанні, зрозуміла, стабільна та значно зручніша за вбудований urllib.

Отримати й підключити бібліотеку було дуже легко — її встановлення займає одну команду, а імпорт виглядає мінімалістично. Почати роботу також було просто: для реалізації потрібного функціоналу достатньо базових методів requests.post() і обробки статус-кодів. Документація цієї бібліотеки є однією з найкраще написаних серед Python-бібліотек: вона чітка, містить приклади та одразу дає уявлення, як правильно формувати запити.

Використання Requests у моєму випадку було інтуїтивним. Формування запиту з JSON-тілом, заголовками та тайм-аутом виконувалось кількома рядками коду. Бібліотека не вимагала додаткових налаштувань чи зайвих допоміжних конструкцій. Поведінка методів була передбачуваною, а взаємодія з іншим моїм кодом — простою.

Певні труднощі виникали лише на етапі обробки помилок, коли сервер міг повернути порожню відповідь або некоректний JSON. З позитивних аспектів можу виділити зручний синтаксис, хорошу документацію, стабільність та гнучкість.

Узагальнення.

В межах даної роботи було виконано вимоги щодо лабораторної роботи №1 (перелічено у розділі з виконаним завданням). Також виконання деякого функціоналу було перекладено на сторонні бібліотеки (самі бібліотеки та звіти про роботу з ними наведено у відповідному розділі). Таким чином, було виконано (можливо частково) умови до лабораторної роботи №3.